T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structures and Algorithms | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 8/26/26 | **DATE SUBMITTED:** | 8/26/26 |
| **NAME:** | Condino, Niel Vincent B. | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

**Prelim Laboratory** **Exam Set D**

## 1. PROCEDURE OUTPUT

```
25  bool isSymmetricStack(std::string input){
26      std::size_t inputSize = input.size();
27      int remainingIndex = 0;
28      int midpoint = inputSize/2;
29
30      if (inputSize <= 1){
31          std::cout << input <<" is symmetric\n";
32          return true;
33      }
34
35      Stack<char> symmetricStack;
36
37      for (int i = 0;i < midpoint;i++){
38          symmetricStack.push(input[i]);
39          remainingIndex++;
40      }
41
42      if (inputSize % 2) {
43          remainingIndex++; //skip middle for odd size
44      }
45
46      for (int i = remainingIndex;i < inputSize ; i++){
47          char item = symmetricStack.top();
48          symmetricStack.pop();
49          if (item != input[i]){
50              std::cout << input << " Not symmetric\n";
51              return false;
52          }
53      }
54
55      std::cout << input <<" is symmetric\n";
56      return true;
57  }
```

**Figure 1:** Array Stack Implementation

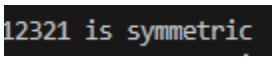
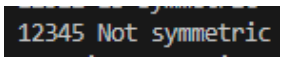
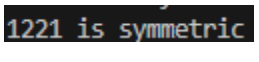
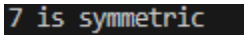
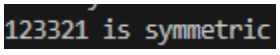
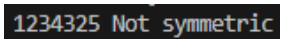
In this approach, an array based stack is used to check the input's symmetry. The program follows this algorithm to achieve the desired output: First, it gets the size of the input, the midpoint (which is the size divided by 2, the integer part of it), and initializes a variable that will keep track of the last index where the first loop will stop which will be named "remainingIndex"; the algorithm then checks if the input has a size of less than or equal to one, if it is then, by nature it is symmetric and the function will exit immediately; If it is not, it will proceed with the implementation of the stack and checking if it is symmetric; It uses for loop to push the characters till it reaches the midpoint. While doing this, the remainingIndex is incremented to keep track of the last index the loop has gone through; If the size of the input's size is an odd number, the remainingIndex is incremented to skip this index as it will not be needed for symmetry checking; The program then evaluates the input's symmetry by comparing the right half of it to the values in the stack. If at least one character is not the same as the one in the stack, the input is considered not symmetric and will return false; If it passes this test, the input is considered symmetric and the program will return true.

```
79  bool isSymmetricQueue(std::string input){
80      std::size_t inputSize = input.size();
81      int midpoint = inputSize/2;
82
83      if (inputSize <= 1){
84          std::cout << input <<" is symmetric\n";
85          return true;
86      }
87
88      //shift midpoint if even
89      if ((inputSize % 2) == 0){
90          midpoint --;
91      }
92
93      qNode<char>* front = nullptr;
94      qNode<char>* back = nullptr;
95
96      //inputsize - 1 to skip null terminators
97      for (int i = inputSize-1;i > midpoint;--i){
98          enqueue(input[i],&front,&back);
99      }
00
01      for (int i = 0;i < midpoint;i++){
02          char item = dequeue(&front,&back);
03          if (item != input[i]){
04              std::cout << input << " Not symmetric\n";
05              //prevent memory leak
06              while (!isQEmpty(front,back)){
07                  dequeue(&front,&back);
08              }
09              return false;
10          }
11      }
12
13      std::cout << input <<" is symmetric\n";
14      return true;
15  }
```

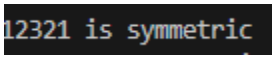
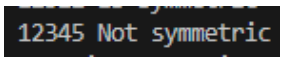
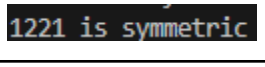
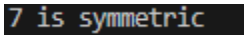
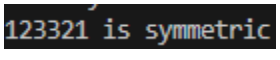
**Figure 2:** Queue linked list implementation

In this approach, a linked list based queue is used to check the input's symmetry. The program follows this algorithm to achieve the desired output: First, it gets the size of the input and the midpoint (which is the size divided by 2, the integer part of it);The algorithm then checks if the input has a size of less than or equal to one, if it is then, by nature it is symmetric and the function will exit immediately; If it is not, it will proceed with the implementation of the stack and checking if it is symmetric; The midpoint is shifted to the left by decrementing it for inputs with an even numbered size. This decision is made so that the designated midpoint stays true, for example the midpoint of a 6 sized input is in index 2 while the raw midpoint is calculated as index 3 which is wrong; Starting from the end, skipping the "null terminator" part of the string, the characters are inserted in a queue. This is chosen so that the program can cross reference it with the other half of the input; The cross reference then happens using a for loop and checking if the characters till midpoint is the same as the data in the queue; If there is one that does not match, the input is declared not symmetric, the program clears the queue to prevent memory leaks, and the program returns false; If it passes the test, the input is considered as symmetric and the function returns true.

**Stack Based**

| Input sequence | Symmetric? | Output | Remark |
|---|---|---|---|
| 1 2 3 2 1 | Yes |   | SUCCESS |
| 1 2 3 4 5 | No |   | SUCCESS |
| 1 2 2 1 | Yes |   | SUCCESS |
| 7 | Yes |   | SUCCESS |
| 1 2 3 3 2 1 | Yes |   | SUCCESS |
| 1 2 3 4 3 2 5 | No |   | SUCCESS |

**Queue Based**

| Input sequence | Symmetric? | Output | Remark |
|---|---|---|---|
| 1 2 3 2 1 | Yes |   | SUCCESS |
| 1 2 3 4 5 | No |   | SUCCESS |
| 1 2 2 1 | Yes |   | SUCCESS |
| 7 | Yes |   | SUCCESS |
| 1 2 3 3 2 1 | Yes |   | SUCCESS |

| | | | |
|---|---|---|---|
| 1 2 3 4 3 2 5 | No | 1234325 Not symmetric | SUCCESS |

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** _____

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** _____

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** _____

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.